UT CHETUMAL

Desarrollo y Gestión de Software

Extracción de conocimiento en bases de datos

Dr. Martín Antonio Santos Romero

T.S.U. Miguel Ángel Peña Pech

9no cuatrimestre

Práctica guiada 4

Algoritmo de regresión

Índice

Introducción 3

Tareas a realizar por el alumno 4

Conclusión 11

Introducción

En esta actividad, desarrollamos un proceso completo de análisis y modelado predictivo utilizando técnicas de regresión lineal en Python. A partir de un conjunto de datos almacenado en un archivo de Excel, se realizó la carga, limpieza y transformación de las variables, así como la codificación de atributos categóricos mediante la técnica de one-hot encoding. Se prepararon los datos para el entrenamiento dividiendo el conjunto en muestras de entrenamiento y prueba, con el fin de evaluar objetivamente el rendimiento del modelo. La actividad se centró en la predicción de precios a partir de diversas características, aplicando métricas estadísticas y visualizaciones que permitieron valorar el ajuste del modelo.

Tareas a realizar por el alumno

Tareas a realizar por el alumno

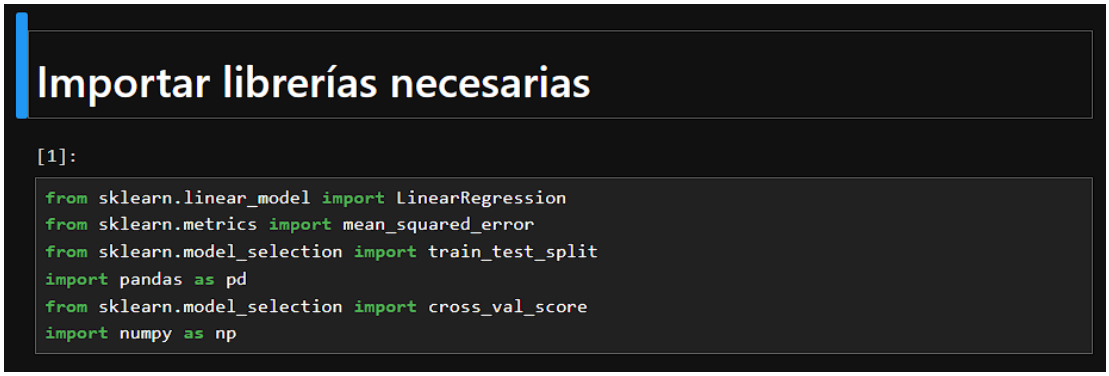
1: Utilizando Jupyter Notebook realiza los ejercicios (laboratorios) del Módulo 4. Funciones, Tuplas, Diccionarios, Excepciones y Procesamiento de Datos; del curso Fundamentos de Python parte 1:

-Crea un nuevo proyecto en Jupyter.
-Guardarlo con el nombre ejerciciosModuloIV_NombreApellidos
-Por cada ejercicio crea una celda nueva de código.

.

Empezamos importando las librerías necesarias para la actividad.

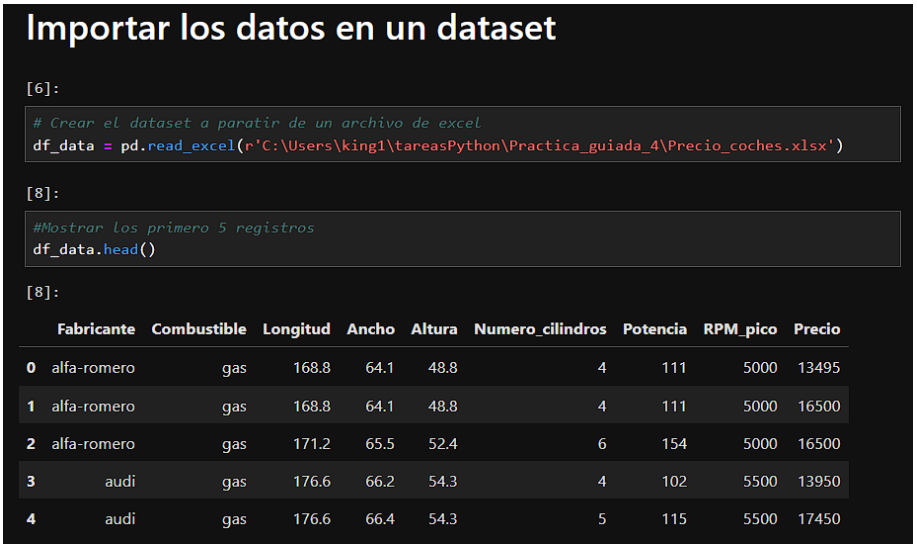
Figura 1.



Fuente: Angel Peña (2025).

Creamos un dataset a partir de un archivo de Excel que guardamos en la carpeta de nuestro proyecto y visualizamos los primeros 5 registros.

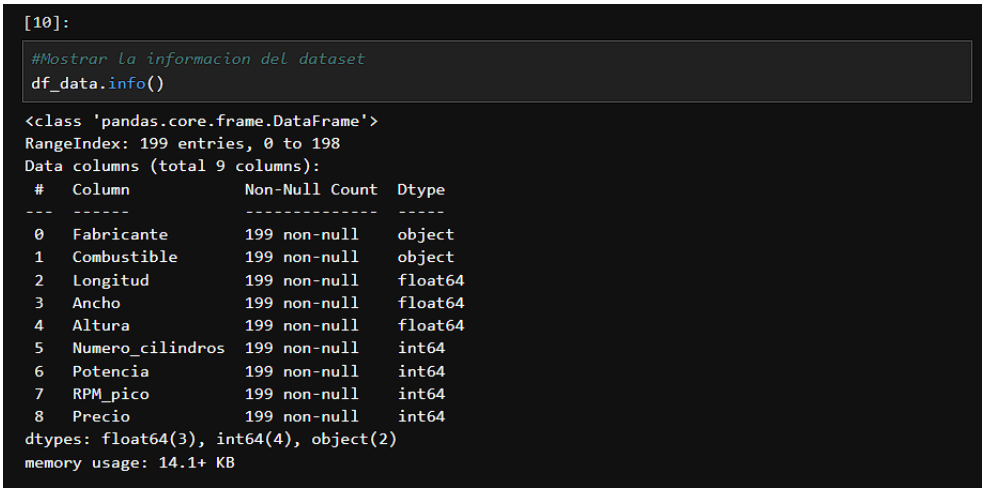
Figura 2.



Fuente: Angel Peña (2025).

Ahora vamos a mostrar la información del dataset con el comando **df_data.info()**

Figura 3.



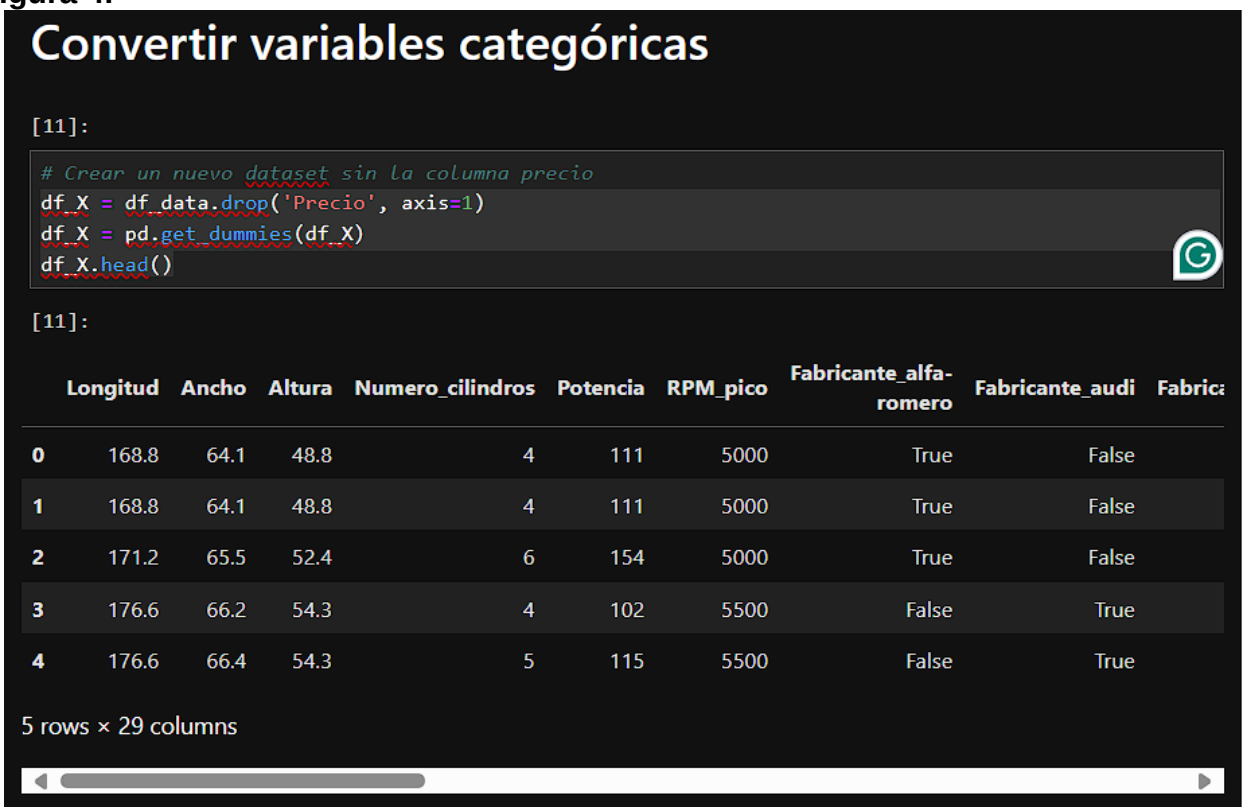
```
[10]:
#Mostrar la informacion del dataset
df_data.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 199 entries, 0 to 198
Data columns (total 9 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Fabricante        199 non-null    object
 1   Combustible       199 non-null    object
 2   Longitud          199 non-null    float64
 3   Ancho             199 non-null    float64
 4   Altura            199 non-null    float64
 5   Numero_cilindros  199 non-null    int64
 6   Potencia          199 non-null    int64
 7   RPM_pico          199 non-null    int64
 8   Precio            199 non-null    int64
dtypes: float64(3), int64(4), object(2)
memory usage: 14.1+ KB
```

Fuente: Angel Peña (2025).

Procedemos a preparar nuestros datos para el análisis. En este paso, eliminamos la columna "Precio" del conjunto de datos original, ya que será la variable objetivo aplicamos la técnica **one-hot encoding** a las columnas categóricas mediante la función **get_dummies()**, lo que nos permite convertir dichas variables en un formato numérico que los modelos de machine learning pueden interpretar.

Figura 4.



Convertir variables categóricas

```
[11]:
# Crear un nuevo dataset sin la columna precio
df_X = df_data.drop('Precio', axis=1)
df_X = pd.get_dummies(df_X)
df_X.head()
```

[11]:

	Longitud	Ancho	Altura	Numero_cilindros	Potencia	RPM_pico	Fabricante_alfa-romero	Fabricante_audi	Fabrica
0	168.8	64.1	48.8	4	111	5000	True	False	
1	168.8	64.1	48.8	4	111	5000	True	False	
2	171.2	65.5	52.4	6	154	5000	True	False	
3	176.6	66.2	54.3	4	102	5500	False	True	
4	176.6	66.4	54.3	5	115	5500	False	True	

5 rows × 29 columns

Fuente: Angel Peña (2025).

Una vez que hemos preparado nuestro conjunto de datos, separamos las variables independientes **(X)** de la variable dependiente **(y)**. En este caso, X contiene todos los valores del nuevo **DataFrame df_X**, es decir, las características que usaremos para predecir, mientras que y contiene los valores de la columna "Precio", que corresponde a la variable que deseamos predecir.
Mostramos los datos:

Figura 5.

Crear un array x con la variables independientes y otro array y para las variables dependientes

[12]:

```
X = df_X.values
y = df_data['Precio'].values
```

[16]:

```
#Array variables independientes
X
```

[16]:

```
array([[168.8, 64.1, 48.8, ..., False, False, True],
       [168.8, 64.1, 48.8, ..., False, False, True],
       [171.2, 65.5, 52.4, ..., False, False, True],
       ...,
       [188.8, 68.9, 55.5, ..., True, False, True],
       [188.8, 68.9, 55.5, ..., True, True, False],
       [188.8, 68.9, 55.5, ..., True, False, True]], dtype=object)
```

[17]:

```
#Array variables dependientes
y
```

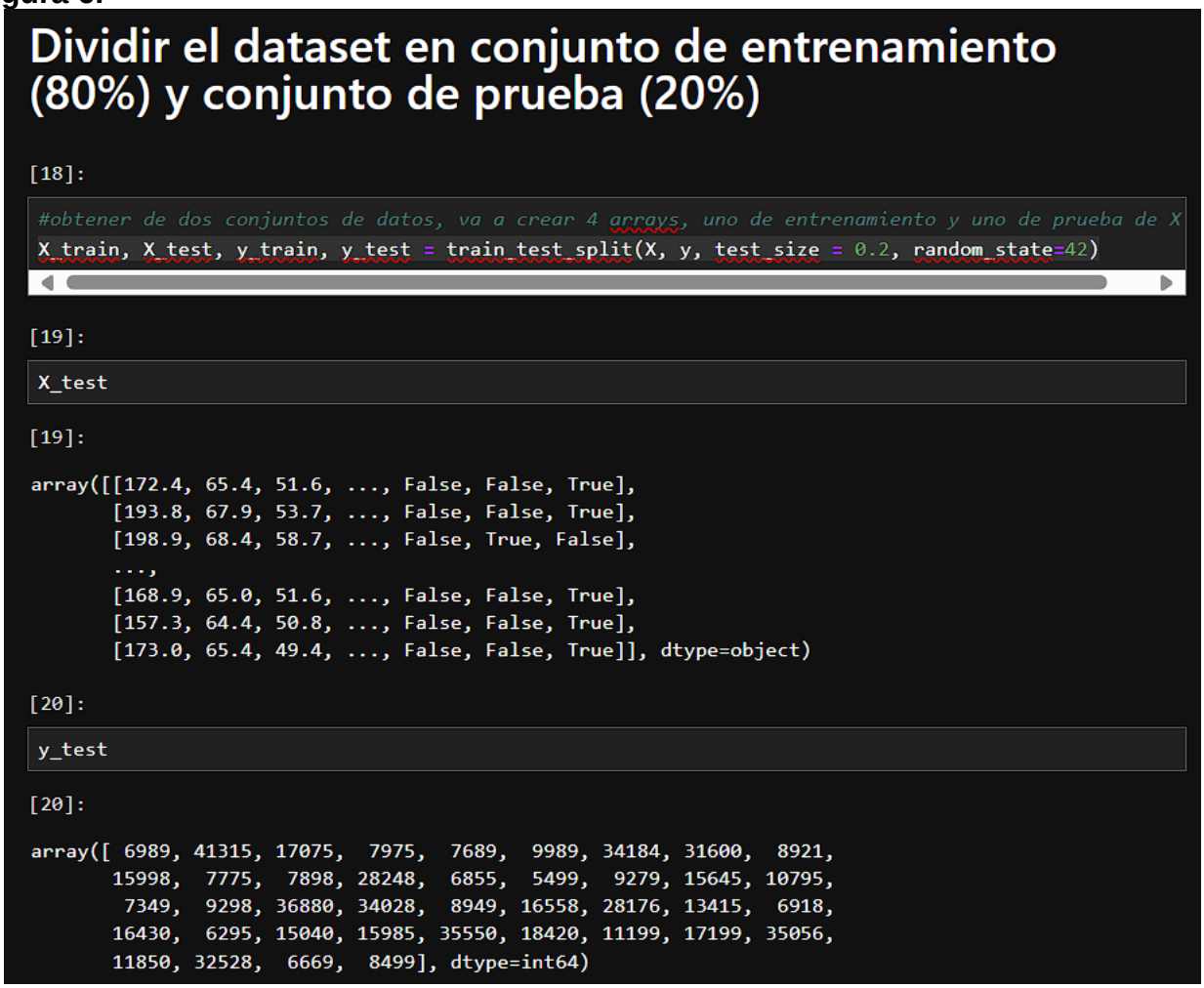
[17]:

```
array([13495, 16500, 16500, 13950, 17450, 15250, 17710, 18920, 23875,
       16430, 16925, 20970, 21105, 24565, 30760, 41315, 36880,  5151,
        6295,  6575,  5572,  6377,  7957,  6229,  6692,  7609,  8558,
        8921, 12964,  6479,  6855,  5399,  6529,  7129,  7295,  7295,
        7895,  9095,  8845, 10295, 12945, 10345,  6785, 11048, 32250,
       35550, 36000,  5195,  6095,  6795,  6695,  7395, 10945, 11845,
       13645, 15645,  8845,  8495, 10595, 10245, 10795, 11245, 18280,
       18344, 25552, 28248, 28176, 31600, 34184, 35056, 40960, 45400,
       16503,  5389,  6189,  6669,  7689,  9959,  8499, 12629, 14869,
       14489,  6989,  8189,  9279,  9279,  5499,  7099,  6649,  6849,
        7349,  7299,  7799,  7499,  7999,  8249,  8949,  9549, 13499,
       14399, 13499, 17199, 19699, 18399, 11900, 13200, 12440, 13860,
       15580, 16900, 16695, 17075, 16630, 17950, 18150,  5572,  7957,
        6229,  6692,  7609,  8921, 12764, 22018, 32528, 34028, 37028,
       11850, 12170, 15040, 15510, 18150, 18620,  5118,  7053,  7603,
        7126,  7775,  9960,  9233, 11259,  7463, 10198,  8013, 11694,
        5348,  6338,  6488,  6918,  7898,  8778,  6938,  7198,  7898,
        7788,  7738,  8358,  9258,  8058,  8238,  9298,  9538,  8449,
        9639,  9989, 11199, 11549, 17669,  8948, 10698,  9988, 10898,
       11248, 16558, 15998, 15690, 15750,  7775,  7975,  7995,  8195,
        8495,  9495,  9995, 11595,  9980, 13295, 13845, 12290, 12940,
       13415, 15985, 16515, 18420, 18950, 16845, 19045, 21485, 22470,
       22625], dtype=int64)
```

Fuente: Angel Peña (2025).

Dividimos nuestro conjunto de datos en dos partes: una para entrenar el modelo **(X_train, y_train)** y otra para evaluar su desempeño **(X_test, y_test)**. Utilizamos la función **train_test_split()** del módulo **sklearn.model_selection**, especificando que el **20%** de los datos se destinarán a la prueba, mientras que el **80%** se utilizarán para el entrenamiento. Además, fijamos el parámetro **random_state=42** para garantizar que la partición sea reproducible.
Mostramos valores:

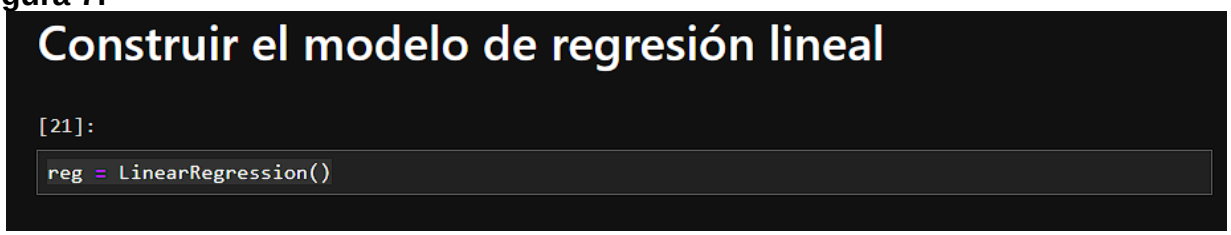
Figura 6.



Fuente: Angel Peña (2025).

Creamos una instancia del modelo de regresión lineal utilizando la clase **LinearRegression()** de la librería **sklearn.linear_model**.

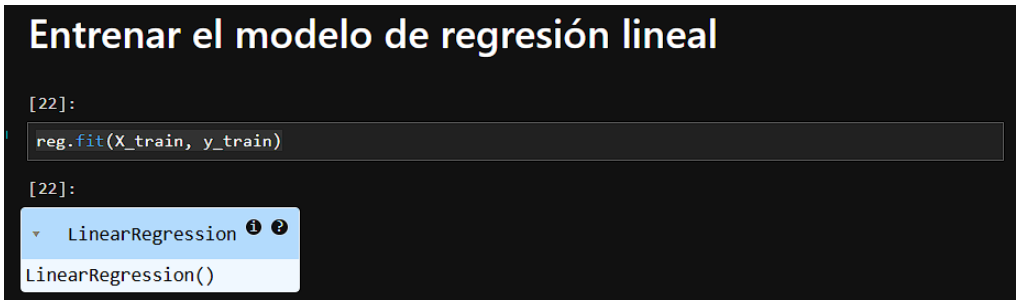
Figura 7.



Fuente: Angel Peña (2025).

Entrenamos el modelo con los datos de entrenamiento mediante el **método .fit()**, utilizando las variables **X_train y y_train**.

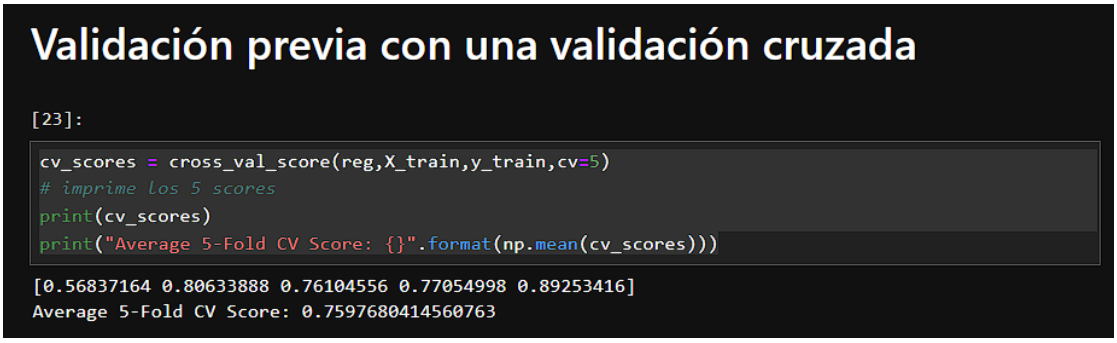
Figura 8.



Fuente: Angel Peña (2025).

Aplicamos una técnica llamada **validación cruzada con 5 divisiones (folds)**. Esta técnica nos permite estimar con mayor precisión el rendimiento del modelo, reduciendo la posibilidad de sobreajuste. Utilizamos la función **cross_val_score()** para evaluar el modelo reg con los datos de entrenamiento.
Esta validación nos devuelve cinco valores que representan la precisión del modelo en cada partición de los datos, además de un promedio que resume su desempeño general.

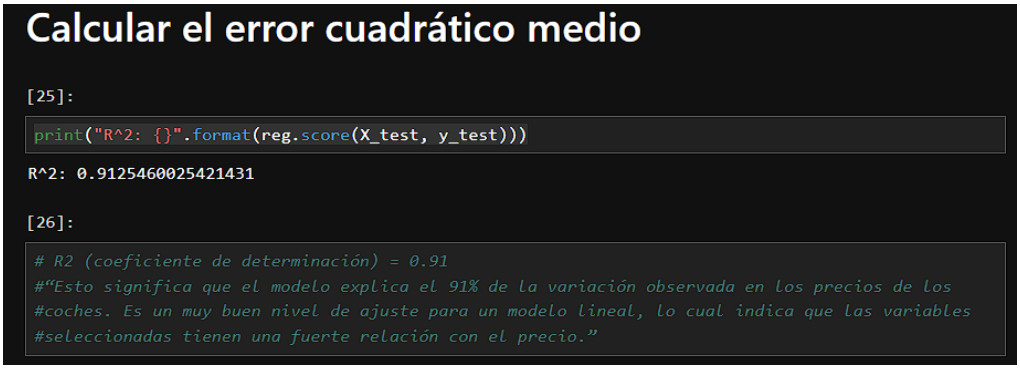
Figura 9.



Fuente: Angel Peña (2025).

Una vez obtenidas las predicciones, evaluamos el rendimiento del modelo utilizando el coeficiente de determinación **R^2**. Este indicador mide qué tan bien se ajustan las predicciones a los valores reales del conjunto de prueba. Un valor cercano a 1 indica un buen ajuste.

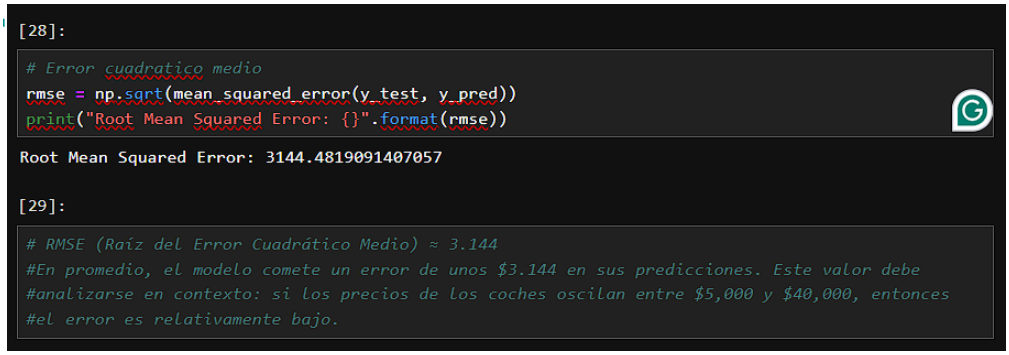
Figura 10.



Fuente: Angel Peña (2025).

Para complementar la evaluación, calculamos el error cuadrático medio **(Root Mean Squared Error)**, el cual indica cuánto difieren en promedio las predicciones del modelo respecto a los valores reales. Este valor se expresa en las mismas unidades que la variable objetivo.
Un valor más bajo de RMSE sugiere que el modelo tiene un mejor rendimiento.

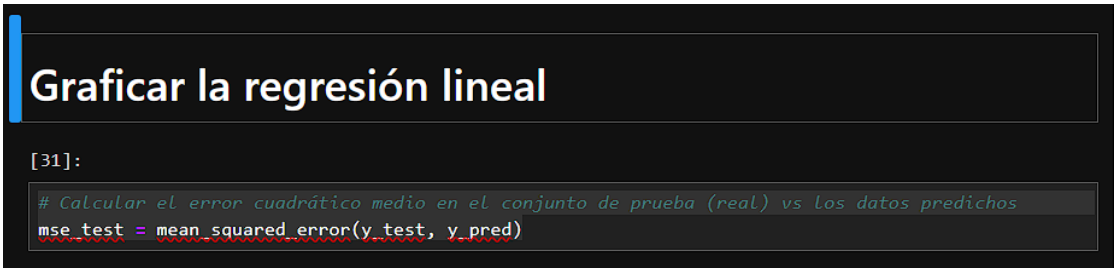
Figura 11.



Fuente: Angel Peña (2025).

También podemos calcular el error cuadrático medio (MSE) de forma directa, para analizar la magnitud del error en su forma no transformada (al cuadrado).
Este valor se utiliza frecuentemente para comparar diferentes modelos, ya que penaliza más fuertemente los errores grandes.

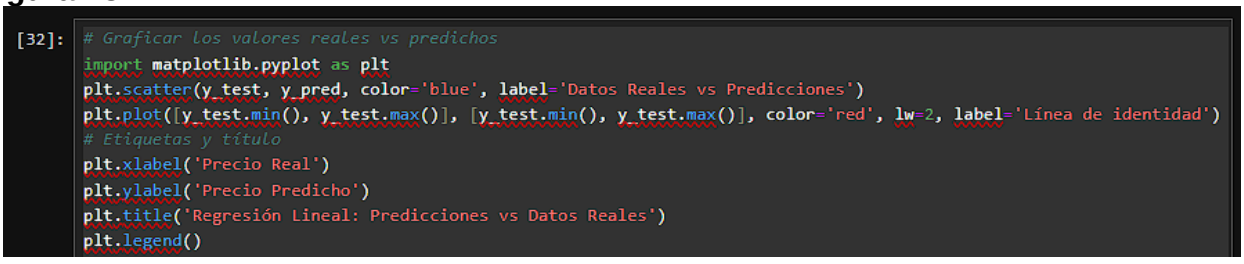
Figura 12.



Fuente: Angel Peña (2025).

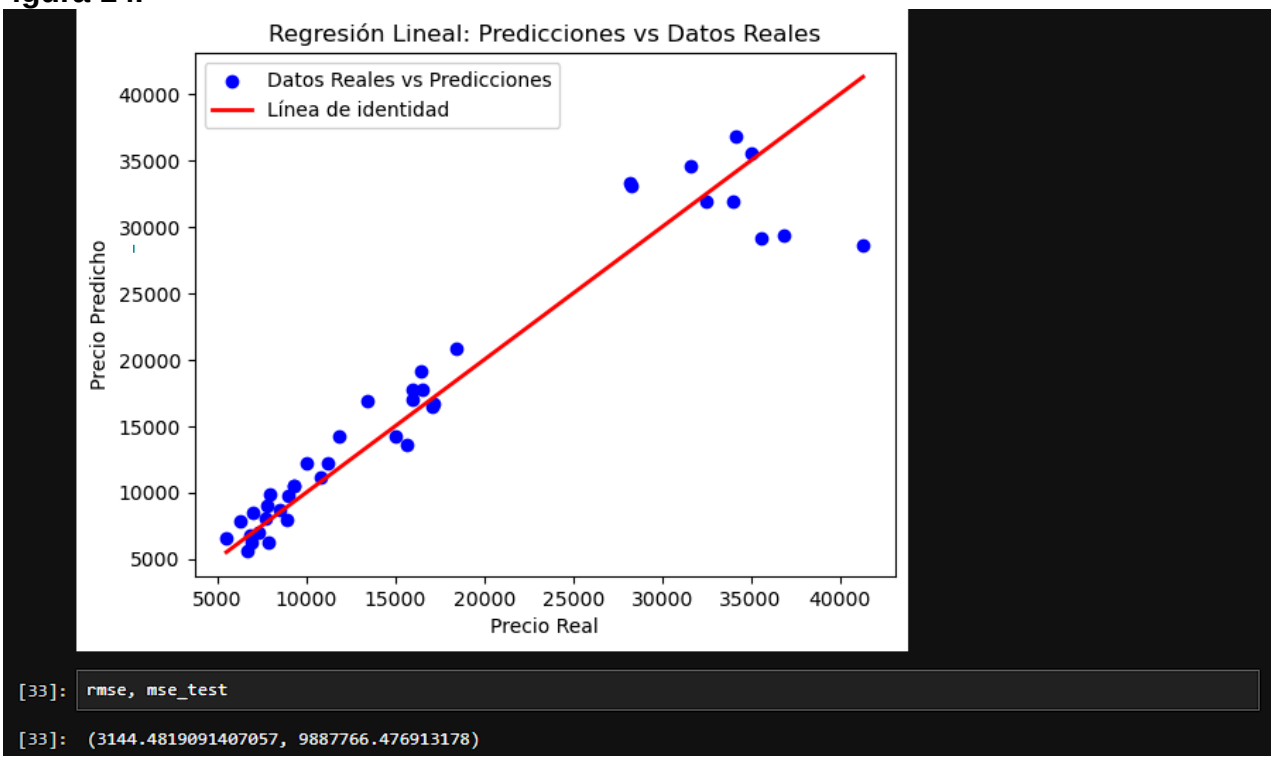
Finalmente, realizamos una visualización que muestra la relación entre los precios reales y los precios predichos. Utilizamos un diagrama de dispersión en donde cada punto representa una predicción. Además, agregamos una línea roja que representa la línea de identidad perfecta (donde el valor predicho sería igual al real).
Esta gráfica permite evaluar visualmente la precisión del modelo: mientras más cerca estén los puntos de la línea roja, mejor será la calidad de las predicciones.

Figura 13.



Fuente: Angel Peña (2025).

Figura 14.



Fuente: Angel Peña (2025).

Conclusión

A través del desarrollo de esta actividad, fue posible comprender el flujo completo de una tarea de aprendizaje supervisado basada en regresión lineal, desde la preparación de los datos hasta la evaluación y validación del modelo. Se aplicaron métricas como el coeficiente de determinación **R^2**, el error cuadrático medio **(MSE)** y su raíz **(RMSE)**, complementadas con una visualización comparativa de los valores reales y predichos. Los resultados obtenidos permiten identificar el grado de precisión del modelo en la estimación de los precios, demostrando así la utilidad de las técnicas de machine learning en la resolución de problemas predictivos en entornos reales. Esta experiencia fortalece el entendimiento de conceptos clave en ciencia de datos y aprendizaje automático.